**EAST WEST UNIVERSITY**
**Department of Computer Science and Engineering**
**CSE430 - Software Testing and Quality Assurance**

---

# Lab Topic: Pytest Unit Testing

## 1. Objective

This lab introduces unit testing in Python using the pytest framework. Students will learn how to write testable Python classes, structure pytest test functions, use fixtures for setup and teardown, and apply parametrization to cover multiple input cases with a single test.

## 2. Learning Outcomes

1. Set up and configure a pytest environment in Python.
2. Implement a Python class (Calculator) with several methods.
3. Write and execute corresponding test functions using pytest.
4. Use pytest fixtures to manage setup and teardown of test objects.
5. Apply @pytest.mark.parametrize to run a test against multiple data sets.
6. Interpret pytest outcomes and maintain structured test documentation.

## 3. Software Requirements

- Python 3.10 or higher.
- pip package manager.
- pytest library.
- A code editor or IDE (VS Code, PyCharm, or similar).

## 4. Lab Procedure

### Step 1 : Creating the Project

1. Create a project folder, e.g. CalculatorApp.
2. Open a terminal inside the folder and create a virtual environment: python -m venv venv.
3. Activate the environment and install pytest: pip install pytest.

### Step 2 : Creating the Source File

1. Inside the project folder, create a file named first.py.
2. Define the Calculator class inside it, as shown in Section 5.

## 5. Implementation under Test

**Source Code : first.py**

```
class Calculator:

    def add(self, a, b):
        return a + b

    def subtract(self, a, b):
        return a - b

    def multiply(self, a, b):
        return a * b

    def divide(self, a, b):
        if b == 0:
            raise ValueError("Cannot divide by zero.")
        return a // b
```

Explanation: each method performs a basic arithmetic operation. The divide() method raises a ValueError when the divisor is zero, which gives pytest an exception to test against.

## 6. Writing the First Test File

Create a file named test_first.py in the same folder. pytest automatically discovers any file matching test_*.py or *_test.py, and within it, any function whose name starts with test_.

**Source Code : test_first.py (basic tests)**

```
import pytest
from first import Calculator

def test_add():
    calc = Calculator()
    assert calc.add(7, 3) == 10, "7 + 3 should equal 10"
    assert calc.add(-3, 3) == 0

def test_subtract():
    calc = Calculator()
    assert calc.subtract(10, 6) == 4
    assert calc.subtract(3, 9) == -6

def test_multiply():
    calc = Calculator()
    assert calc.multiply(3, 5) == 15
    assert calc.multiply(0, 100) == 0

def test_divide():
    calc = Calculator()
    assert calc.divide(10, 5) == 2
    assert calc.divide(9, 3) == 3


def test_divide_by_zero():
    calc = Calculator()
    with pytest.raises(ValueError) as excinfo:
```

```
        calc.divide(10, 0)
    assert str(excinfo.value) == "Cannot divide by zero."


def test_multiple_assertions():
    calc = Calculator()
    assert calc.add(2, 3) == 5
    assert calc.multiply(2, 3) > 5
    assert calc.subtract(7, 3) != 10
```

## 7. Running the Tests

- Run all tests in the folder: pytest
- Run a single file: pytest test_first.py
- Run a single function: pytest test_first.py::test_add
- Verbose output (shows each test name and result): pytest -v

pytest prints a dot for each passing test and an F for each failure when run without -v, then a summary line. With -v, it lists every test function name with PASSED or FAILED next to it. A failed assert shows the exact values on both sides of the comparison, which is usually enough to diagnose the problem without extra print statements.

## 8. Fixtures: Setup and Teardown

Repeating Calculator() in every test function works, but it duplicates setup code. A fixture moves that setup into one place and hands the result to any test that asks for it by name. Anything after yield runs as a teardown, once the test using it has finished.

**Source Code : test_fixture.py**

```
import logging
import pytest

logging.basicConfig(level=logging.INFO)

from first import Calculator

@pytest.fixture
def calculator_fixture():
    print("\nSetup: creating calculator")
    calc = Calculator()
    yield calc
    print("\nAfter: destroying calculator")

def test_add(calculator_fixture):
    assert calculator_fixture.add(2, 3) == 5

def test_subtract(calculator_fixture):
    assert calculator_fixture.subtract(10, 5) == 5

def test_multiply(calculator_fixture):
    assert calculator_fixture.multiply(2, 3) == 6
```

A test receives the fixture by naming it as a parameter; pytest matches the name and injects the value. The default scope is “function”, so the fixture is rebuilt for every test. scope="module" or scope="session" can be passed to @pytest.fixture to share one instance across many tests instead.

## 9. Parametrized Tests

@pytest.mark.parametrize replaces several near-identical test functions with one function and a list of input/output pairs. pytest runs the function once per row in the list and reports each run as a separate test.

**Source Code : test_parametrize.py**

```
import pytest
from first import Calculator

@pytest.mark.parametrize(
    "a, b, expected",
    [
        (7, 3, 10),
        (-3, 3, 0),
        (0, 0, 0),
        (100, 200, 300),
    ],
)
def test_add_parameterized(a, b, expected):
    calc = Calculator()
    print(f"Testing add({a}, {b}) == {expected}")
    assert calc.add(a, b) == expected
```

Run with pytest -v to see each parameter set listed as its own test, e.g. test_add_parameterized[7-3-10]. This is the pytest equivalent of writing four separate test methods, without the repetition.

## 10. Test Case Documentation

| Method | Inputs | Expected Output | Description |
|---|---|---|---|
| add | 5, 3 | 8 | Adding positive numbers |
| add | -2, 2 | 0 | Mixed sign addition |
| subtract | 9, 5 | 4 | Basic subtraction |
| multiply | 4, 0 | 0 | Multiplication by zero |
| divide | 6, 3 | 2 | Simple division |
| divide | 7, 0 | ValueError | Division by zero case |

## 11. Lab Tasks

### Lab Task 1 : Grade Classifier

Objective: test boundary logic that maps a numeric score to a letter grade.

**Implementation (grade_classifier.py)**

```
class GradeClassifier:
    def letter_grade(self, score):
        if score < 0 or score > 100:
            raise ValueError("Score must be between 0 and 100.")
        if score >= 90:
```

```
        return "A"
    if score >= 80:
        return "B"
    if score >= 70:
        return "C"
    if score >= 60:
        return "D"
    return "F"
```

**Test ideas (test_grade_classifier.py)**

- 90 → “A”, 89 → “B”, 60 → “D”, 59 → “F” (boundary values matter here).
- 105 or -1 → expect ValueError.

Required: write this test using @pytest.mark.parametrize, with the score and expected letter as the two parameters, instead of one function per grade band.

## Lab Task 2 : Library Book Loan

Objective: test state changes and exceptional conditions on borrowing and returning a book.

**Implementation (library.py)**

```
class Book:
    def __init__(self, title):
        self.title = title
        self.is_borrowed = False

    def borrow(self):
        if self.is_borrowed:
            raise RuntimeError("Book is already borrowed.")
        self.is_borrowed = True

    def return_book(self):
        if not self.is_borrowed:
            raise RuntimeError("Book was not borrowed.")
        self.is_borrowed = False
```

**Test ideas**

- Borrow an available book → is_borrowed becomes True.
- Borrow an already-borrowed book → expect RuntimeError.
- Return a borrowed book → is_borrowed becomes False.
- Return a book that was never borrowed → expect RuntimeError.

Required: use a fixture that creates and returns a fresh Book instance, and have every test function take that fixture as a parameter.

## Lab Task 3 : Playlist Queue

Objective: test a small FIFO structure and its edge cases.

**Implementation (playlist.py)**

```
class Playlist:
    def __init__(self):
        self.tracks = []
```

```
    def add_track(self, name):
        self.tracks.append(name)

    def play_next(self):
        if not self.tracks:
            raise IndexError("Playlist is empty.")
        return self.tracks.pop(0)

    def track_count(self):
        return len(self.tracks)
```

**Test ideas**

- Add two tracks, play_next() returns the first one added, in order.
- track_count() reflects additions and removals correctly.
- Calling play_next() on an empty playlist → expect IndexError.

No fixture or parametrize requirement here : write this one with plain test functions, the way Section 6 did.

## Lab Task 4 : Unit Price Comparator

Objective: test a small calculation function across several input combinations.

**Implementation (price_compare.py)**

```
class PriceComparator:
    def unit_price(self, total_price, quantity):
        if quantity <= 0:
            raise ValueError("Quantity must be positive.")
        return total_price / quantity

    def cheaper_unit(self, price_a, qty_a, price_b, qty_b):
        unit_a = self.unit_price(price_a, qty_a)
        unit_b = self.unit_price(price_b, qty_b)
        return "A" if unit_a < unit_b else "B"
```

**Test ideas**

- unit_price(10, 4) is approximately 2.5 : use pytest.approx() since this is a float result.
- quantity = 0 → expect ValueError.
- cheaper_unit() picks the correct option across at least three different price/quantity combinations.

Required: write the cheaper_unit() tests using @pytest.mark.parametrize, with price_a, qty_a, price_b, qty_b, and expected as the parameter columns.

## Lab Task 5 : Parking Fee Calculator

Objective: combine multiple checks and a shared fixture to model a small billing rule.

**Implementation (parking.py)**

```
class ParkingFeeCalculator:
    def __init__(self, hourly_rate=20):
        self.hourly_rate = hourly_rate

    def calculate_fee(self, hours):
```

```
    if hours < 0:
        raise ValueError("Hours cannot be negative.")
    if hours == 0:
        return 0
    full_hours = int(hours) + (1 if hours % 1 > 0 else 0)
    return full_hours * self.hourly_rate
```

**Test ideas**

- 0 hours → fee = 0.
- 2 hours → fee = 2 × hourly_rate.
- 2.5 hours → rounds up to 3 full hours.
- Negative hours → expect ValueError.

Required: use a fixture that returns a ParkingFeeCalculator with hourly_rate=20, and have each test request it as a parameter.

## 12. Homework

### Homework 1 :Password Strength Checker

Objective: build and test a small rule-based validator.

**Tasks**

1. Implement a PasswordChecker class with is_strong(password), which returns True only if the password is at least 8 characters long, contains at least one digit, and contains at least one uppercase letter. Anything else returns False.
2. Write pytest tests covering: a strong password, one that's too short, one with no digit, one with no uppercase letter, and an empty string.
3. Write at least one of those tests using @pytest.mark.parametrize, with the password string and the expected boolean as the two parameters.
4. Use a fixture that returns a fresh PasswordChecker() instance and have every test function take it as a parameter.
5. Submit: the source file, the test file, and a short reflection (about 100 words) on which rule was hardest to test correctly.

### Homework 2 :Movie Ticket Booking

Objective: model a small booking rule with both state and exceptions.

**Tasks**

1. Implement a Cinema class with total_seats in its constructor, book_seats(n), available_seats(), and is_sold_out().
- book_seats(n) should raise ValueError if n is greater than the seats currently available.
- is_sold_out() should return True once available_seats() reaches zero.
2. Write pytest tests using pytest.raises for the overbooking case, and plain assert statements for the True/False checks.
3. Add at least five distinct test cases: a normal booking, an exact sellout, an overbooking attempt, booking zero seats, and one more case of your choosing.
4. Submit a test-case table alongside your source files.

## Homework 3 :Inventory Restock Tracker

Objective: apply pytest to a small stock-management class with several related checks.

**Tasks**

1. Implement an InventoryItem class with quantity, reorder_level in its constructor, restock(amount), sell(amount), and needs_reorder().

Write pytest tests that include:

- sell(amount) raising ValueError when amount exceeds the current quantity.
- needs_reorder() returning True once quantity drops to or below reorder_level.
- Several restock/sell combinations, written using @pytest.mark.parametrize, checking the resulting quantity.

2. Give each test function a clear, descriptive name rather than a generic one like test_1.
3. Document results in a test-case summary table and add one observation about an edge case you found while testing.

## 13. Submission Format

- Source code files (.py for implementation and tests).
- Screenshot of a successful pytest run (pytest -v output).
- Test-case table (method, input, expected output, result).
- Reflection paragraph (what you learned or would improve).

## 14. Conclusion

pytest gives Python the same systematic approach to validation that JUnit gives Java, with less boilerplate: plain assert statements replace assertEquals(), fixtures replace @BeforeEach/@AfterEach, and parametrize covers in one function what would otherwise need several. The underlying discipline carries over directly : isolate one behavior per test, cover the exception paths, and keep a written record of what each test checks.